MTE Fixed-Length Add-On Developer's Guide

MTE Version 2.1

Table of Contents

1 Introduction

2 Terms and Abbreviations

3 Prerequisites

4 Caveats

5 Architecture

6 SDK Structure

7 Options

- 7.1 Choosing the Fixed Length

8 Status

9 Basic C API

- 9.1 Overview
  - 9.1.1 Encoder
- 9.2 Files
- 9.3 Functions
  - 9.3.1 Encoder
    - 9.3.1.1 `mte_flen_enc_state_bytes` (C)
    - 9.3.1.2 `mte_flen_enc_state_bytes_d` (C)
    - 9.3.1.3 `mte_flen_enc_state_init` (C)
    - 9.3.1.4 `mte_flen_enc_state_init_d` (C)
    - 9.3.1.5 `mte_flen_enc_instantiate` (C)
    - 9.3.1.6 `mte_flen_enc_reseed_counter` (C)
    - 9.3.1.7 `mte_flen_enc_save_bytes` (C)
    - 9.3.1.8 `mte_flen_enc_save_bytes_b64` (C)
    - 9.3.1.9 `mte_flen_enc_state_save` (C)
    - 9.3.1.10 `mte_flen_enc_state_save_b64` (C)
    - 9.3.1.11 `mte_flen_enc_state_restore` (C)
    - 9.3.1.12 `mte_flen_enc_state_restore_b64` (C)
    - 9.3.1.13 `mte_flen_enc_buff_bytes` (C)
    - 9.3.1.14 `mte_flen_enc_buff_bytes_b64` (C)
    - 9.3.1.15 `mte_flen_enc_encode` (C)
    - 9.3.1.16 `mte_flen_enc_encode_b64` (C)
    - 9.3.1.17 `mte_flen_enc_uninstantiate` (C)

10 C Support API

- 10.1 Overview
  - 10.1.1 Encoder
- 10.2 Files
- 10.3 Functions
  - 10.3.1 Encoder Support
    - 10.3.1.1 `mte_supp_flen_enc_alloc` (C)
    - 10.3.1.2 `mte_supp_flen_enc_state_save` (C)
    - 10.3.1.3 `mte_supp_flen_enc_state_save_b64` (C)
    - 10.3.1.4 `mte_supp_flen_enc_encode` (C)

10.3.1.5 `mte_supp_flen_enc_encode_b64` (C)
10.3.1.6 `mte_supp_flen_enc_free` (C)

11 C++ Language Interface

11.1 Overview

11.1.1 Encoder

11.2 Files

11.3 Types

11.3.1 Classes

11.3.1.1 `MteFlenEnc`

11.4 Functions

11.4.1 Encoder

11.4.1.1 `MteFlenEnc` Constructor (default options) (CPP)
11.4.1.2 `MteFlenEnc` Constructor (runtime options) (CPP)
11.4.1.3 `MteFlenEnc` Destructor (CPP)
11.4.1.4 `MteFlenEnc::instantiate` (`void *`) (CPP)
11.4.1.5 `MteFlenEnc::instantiate` (`std::string`) (CPP)
11.4.1.6 `MteFlenEnc::getReseedCounter` (CPP)
11.4.1.7 `MteFlenEnc::saveState` (CPP)
11.4.1.8 `MteFlenEnc::saveStateB64` (CPP)
11.4.1.9 `MteFlenEnc::restoreState` (CPP)
11.4.1.10 `MteFlenEnc::restoreStateB64` (CPP)
11.4.1.11 `MteFlenEnc::getBuffBytes` (CPP)
11.4.1.12 `MteFlenEnc::getBuffBytesB64` (CPP)
11.4.1.13 `MteFlenEnc::encode` (`void *`) (CPP)
11.4.1.14 `MteFlenEnc::encodeB64` (`void *`) (CPP)
11.4.1.15 `MteFlenEnc::encode` (`std::string`) (CPP)
11.4.1.16 `MteFlenEnc::encodeB64` (`std::string`) (CPP)
11.4.1.17 `MteFlenEnc::encode` (external buffers) (CPP)
11.4.1.18 `MteFlenEnc::encodeB64` (external buffers) (CPP)
11.4.1.19 `MteFlenEnc::uninstantiate` (CPP)

12 C# Language Interface

12.1 Overview

12.1.1 Encoder

12.2 Files

12.3 Types

12.3.1 Classes

12.3.1.1 `MteFlenEnc`

12.4 Functions

12.4.1 Encoder

12.4.1.1 `MteFlenEnc` Constructor (default options) (CS)
12.4.1.2 `MteFlenEnc` Constructor (interop, default options) (CS)
12.4.1.3 `MteFlenEnc` Constructor (runtime options) (CS)
12.4.1.4 `MteFlenEnc` Constructor (interop, runtime options) (CS)
12.4.1.5 `MteFlenEnc` Destructor (CS)
12.4.1.6 `MteFlenEnc.Instantiate` (byte[]) (CS)
12.4.1.7 `MteFlenEnc.Instantiate` (`string`) (CS)
12.4.1.8 `MteFlenEnc.GetReseedCounter` (CS)
12.4.1.9 `MteFlenEnc.SaveState` (CS)

12.4.1.10 `MteFlenEnc.SaveStateB64` (CS)
12.4.1.11 `MteFlenEnc.RestoreState` (CS)
12.4.1.12 `MteFlenEnc.RestoreStateB64` (CS)
12.4.1.13 `MteFlenEnc.GetBuffBytes` (CS)
12.4.1.14 `MteFlenEnc.GetBuffBytesB64` (CS)
12.4.1.15 `MteFlenEnc.Encode` (`byte[]`) (CS)
12.4.1.16 `MteFlenEnc.EncodeB64` (`byte[]`) (CS)
12.4.1.17 `MteFlenEnc.Encode` (`string`) (CS)
12.4.1.18 `MteFlenEnc.EncodeB64` (`string`) (CS)
12.4.1.19 `MteFlenEnc.Encode` (external buffers) (CS)
12.4.1.20 `MteFlenEnc.EncodeB64` (external buffers) (CS)
12.4.1.21 `MteFlenEnc.Uninstantiate` (CS)

13 Go Language Interface
13.1 Overview
13.1.1 Encoder
13.2 Files
13.3 Types
13.3.1 Structures
13.3.1.1 `FlenEnc`
13.4 Functions
13.4.1 Encoder
13.4.1.1 `NewFlenEncDef` (Go)
13.4.1.2 `NewFlenEncOpt` (Go)
13.4.1.3 `(*FlenEnc)Destroy` (Go)
13.4.1.4 `(*FlenEnc)Instantiate` (Go)
13.4.1.5 `(*FlenEnc)InstantiateStr` (Go)
13.4.1.6 `(*FlenEnc)GetReseedCounter` (Go)
13.4.1.7 `(*FlenEnc)SaveState` (Go)
13.4.1.8 `(*FlenEnc)SaveStateB64` (Go)
13.4.1.9 `(*FlenEnc)RestoreState` (Go)
13.4.1.10 `(*FlenEnc)RestoreStateB64` (Go)
13.4.1.11 `(*FlenEnc)Encode` (Go)
13.4.1.12 `(*FlenEnc)EncodeB64` (Go)
13.4.1.13 `(*FlenEnc)EncodeStr` (Go)
13.4.1.14 `(*FlenEnc)EncodeStrB64` (Go)
13.4.1.15 `(*FlenEnc)Uninstantiate` (Go)

14 Java Language Interface
14.1 Overview
14.1.1 Encoder
14.2 Files
14.3 Types
14.3.1 Classes
14.3.1.1 `MteFlenEnc`
14.4 Functions
14.4.1 Encoder
14.4.1.1 `MteFlenEnc` Constructor (default options) (Java)
14.4.1.2 `MteFlenEnc` Constructor (runtime options) (Java)
14.4.1.3 `MteFlenEnc.instantiate` (`byte[]`) (Java)

14.4.1.4 `MteFlenEnc.instantiate` (`String`) (Java)
14.4.1.5 `MteFlenEnc.getReseedCounter` (Java)
14.4.1.6 `MteFlenEnc.saveState` (Java)
14.4.1.7 `MteFlenEnc.saveStateB64` (Java)
14.4.1.8 `MteFlenEnc.restoreState` (Java)
14.4.1.9 `MteFlenEnc.restoreStateB64` (Java)
14.4.1.10 `MteFlenEnc.getBuffBytes` (Java)
14.4.1.11 `MteFlenEnc.getBuffBytesB64` (Java)
14.4.1.12 `MteFlenEnc.encode` (`byte[]`) (Java)
14.4.1.13 `MteFlenEnc.encodeB64` (`byte[]`) (Java)
14.4.1.14 `MteFlenEnc.encode` (`String`) (Java)
14.4.1.15 `MteFlenEnc.encodeB64` (`String`) (Java)
14.4.1.16 `MteFlenEnc.encode` (external buffers) (Java)
14.4.1.17 `MteFlenEnc.encodeB64` (external buffers) (Java)
14.4.1.18 `MteFlenEnc.uninstantiate` (Java)

15 Objective-C Language Interface
15.1 Overview
15.1.1 Encoder
15.2 Files
15.3 Types
15.3.1 Classes
15.3.1.1 `MteFlenEnc`
15.4 Functions
15.4.1 Encoder
15.4.1.1 `MteFlenEnc -initWithFixedBytes:` (ObjC)
15.4.1.2 `MteFlenEnc -initWithDrbg:tokBytes:verifiers:fixedBytes:` (ObjC)
15.4.1.3 `MteFlenEnc -dealloc` (ObjC)
15.4.1.4 `MteFlenEnc -instantiate:bytes:` (ObjC)
15.4.1.5 `MteFlenEnc -instantiate:` (ObjC)
15.4.1.6 `MteFlenEnc -getReseedCounter` (ObjC)
15.4.1.7 `MteFlenEnc -saveState:` (ObjC)
15.4.1.8 `MteFlenEnc -saveStateB64` (ObjC)
15.4.1.9 `MteFlenEnc -restoreState:` (ObjC)
15.4.1.10 `MteFlenEnc -restoreStateB64:` (ObjC)
15.4.1.11 `MteFlenEnc -getBuffBytes` (ObjC)
15.4.1.12 `MteFlenEnc -getBuffBytesB64` (ObjC)
15.4.1.13 `MteFlenEnc -encode:dataBytes:encodedBytes:status:` (ObjC)
15.4.1.14 `MteFlenEnc -encodeB64:dataBytes:status:` (ObjC)
15.4.1.15 `MteFlenEnc -encode:encodedBytes:status:` (ObjC)
15.4.1.16 `MteFlenEnc -encodeB64:status:` (ObjC)
15.4.1.17 `MteFlenEnc -encode:dataOff:dataBytes:encoded:encOff:encBytes` (ObjC)
15.4.1.18 `MteFlenEnc -encodeB64:dataOff:dataBytes:encoded:encOff:encBytes` (ObjC)
15.4.1.19 `MteFlenEnc -uninstantiate` (ObjC)

16 Python Language Interface
16.1 Overview
16.1.1 Encoder
16.2 Files
16.3 Types

16.3.1 Classes
16.3.1.1 `MteFlenEnc`
16.4 Functions
16.4.1 Encoder
16.4.1.1 `MteFlenEnc` Initializer (Python)
16.4.1.2 `MteFlenEnc.fromdefault` (Python)
16.4.1.3 `MteFlenEnc` Delete (Python)
16.4.1.4 `MteFlenEnc.instantiate` (Python)
16.4.1.5 `MteFlenEnc.get_reseed_counter` (Python)
16.4.1.6 `MteFlenEnc.save_state` (Python)
16.4.1.7 `MteFlenEnc.save_state_b64` (Python)
16.4.1.8 `MteFlenEnc.restore_state` (Python)
16.4.1.9 `MteFlenEnc.restore_state_b64` (Python)
16.4.1.10 `MteFlenEnc.encode` (Python)
16.4.1.11 `MteFlenEnc.encode_b64` (Python)
16.4.1.12 `MteFlenEnc.uninstantiate` (Python)
17 Swift Language Interface
17.1 Overview
17.1.1 Encoder
17.2 Files
17.3 Types
17.3.1 Classes
17.3.1.1 `MteFlenEnc`
17.4 Functions
17.4.1 Encoder
17.4.1.1 `MteFlenEnc` Initializer (default options) (Swift)
17.4.1.2 `MteFlenEnc` Initializer (runtime options) (Swift)
17.4.1.3 `MteFlenEnc` Deinitializer (Swift)
17.4.1.4 `MteFlenEnc.instantiate` (`[UInt8]`) (Swift)
17.4.1.5 `MteFlenEnc.instantiate` (`String`) (Swift)
17.4.1.6 `MteFlenEnc.getReseedCounter` (Swift)
17.4.1.7 `MteFlenEnc.saveState` (Swift)
17.4.1.8 `MteFlenEnc.saveStateB64` (Swift)
17.4.1.9 `MteFlenEnc.restoreState` (Swift)
17.4.1.10 `MteFlenEnc.restoreStateB64` (Swift)
17.4.1.11 `MteFlenEnc.getBuffBytes` (Swift)
17.4.1.12 `MteFlenEnc.getBuffBytesB64` (Swift)
17.4.1.13 `MteFlenEnc.encode` (`[UInt8]`) (Swift)
17.4.1.14 `MteFlenEnc.encodeB64` (`[UInt8]`) (Swift)
17.4.1.15 `MteFlenEnc.encode` (`String`) (Swift)
17.4.1.16 `MteFlenEnc.encodeB64` (`String`) (Swift)
17.4.1.17 `MteFlenEnc.encode` (external buffers) (Swift)
17.4.1.18 `MteFlenEnc.encodeB64` (external buffers) (Swift)
17.4.1.19 `MteFlenEnc.uninstantiate` (Swift)
18 WASM Language Interface
18.1 Overview
18.1.1 Encoder
18.2 Files

18.3 Types
18.3.1 Classes
18.3.1.1 `MteFlenEnc`
18.4 Functions
18.4.1 Encoder
18.4.1.1 `MteFlenEnc` Constructor (WASM)
18.4.1.2 `MteFlenEnc.fromdefault` (WASM)
18.4.1.3 `MteFlenEnc.destruct` (WASM)
18.4.1.4 `MteFlenEnc.instantiate` (WASM)
18.4.1.5 `MteFlenEnc.getReseedCounter` (WASM)
18.4.1.6 `MteFlenEnc.saveState` (WASM)
18.4.1.7 `MteFlenEnc.saveStateB64` (WASM)
18.4.1.8 `MteFlenEnc.restoreState` (WASM)
18.4.1.9 `MteFlenEnc.restoreStateB64` (WASM)
18.4.1.10 `MteFlenEnc.encode` (WASM)
18.4.1.11 `MteFlenEnc.encodeB64` (WASM)
18.4.1.12 `MteFlenEnc.encodeStr` (WASM)
18.4.1.13 `MteFlenEnc.encodeStrB64` (WASM)
18.4.1.14 `MteFlenEnc.uninstantiate` (WASM)
19 **Contact Eclypses**

1 Introduction

The Fixed-Length Add-On is a replacement for the core encoder that ensures all messages are encoded to a fixed length. If the input is too long, it is truncated to the fixed length; if the input is too short, it is padded up to the fixed length. The padding consists of bytes with value `0` ("null bytes"). The null bytes are appended to the input data up to the fixed length.

The core encoder is used by the fixed length encoder, so it is important to understand the core encoder first. Refer to the *MTE Core Developer's Guide*.

The core decoder is used with the fixed length encoder. No length information is transmitted, so it is up to the decoding application to have a way to know what is valid. In many cases, this add-on is used to encode variable-length strings, and null bytes are used for padding, and therefore indicate the end of the string. If you are not encoding strings, it is up to you to use appropriate logic to determine the valid decoded length.

2 Terms and Abbreviations

Refer to the *MTE Core Developer's Guide* terms and abbreviations.

3 Prerequisites

Refer to the *MTE Core Developer's Guide* prerequisites.

4 Caveats

Refer to the *MTE Core Developer's Guide* caveats.

5 Architecture

Refer to the *MTE Core Developer's Guide* architecture.

6 SDK Structure

Refer to the *MTE Core Developer's Guide* SDK structure.

7 Options

The MTE Fixed-Length Add-On uses the core options documented in the *MTE Core Developer's Guide*, as well as the following:

Option	Description
Fixed Length	The desired *input* fixed length. Inputs longer than this are truncated to this length; inputs shorter than this are padded to this length. The result will be fixed-length messages, but that length depends on the other options chosen.

7.1 Choosing the Fixed Length

To choose the fixed length, determine the maximum allowed input size. You may need to consider adding a length to the data if the padding cannot imply the valid length, and this may increase the fixed length necessary.

8 Status

Refer to the *MTE Core Developer's Guide* status.

9 Basic C API

9.1 Overview

9.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and destroy it.

To create, get the state size with `mte_flen_enc_state_bytes()`, initialize the state with `mte_flen_enc_state_init()`, and instantiate with `mte_flen_enc_instantiate()`.

To encode, use `mte_flen_enc_encode()` or `mte_flen_enc_encode_b64()`.

To destroy, use `mte_flen_enc_uninstantiate()`.

9.2 Files

The MTE Fixed-Length Add-On uses the core header files (other than `mte_enc.h`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
mte_flen_enc.h	This defines the Fixed-Length Add-On encoder functions.

9.3 Functions

9.3.1 Encoder

Encoder-specific functions are declared in `mte_flen_enc.h`.

9.3.1.1 `mte_flen_enc_state_bytes` (C)

```
uint32_t mte_flen_enc_state_bytes(mte_drbgs drbg, uint32_t tok_bytes, mte_verifiers
verifiers)
```

Returns the encoder state size for the given DRBG algorithm, token size in bytes, and verifiers algorithm. Returns `0` if the combination of arguments is not usable. Use this function when an MTE-provided DRBG is going to be used. The associated initialization function is `mte_flen_enc_state_init()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_flen_enc_state_bytes_d()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.

9.3.1.2 `mte_flen_enc_state_bytes_d` (C)

```
uint32_t mte_flen_enc_state_bytes_d(uint32_t tok_bytes, mte_verifiers verifiers)
```

Returns the encoder state size for the given token size in bytes, and verifiers algorithm. Returns `0` if the combination of arguments is not usable. Use this function when an MTE-provided DRBG is not going to be used. The associated initialization function is `mte_flen_enc_state_init_d()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.

9.3.1.3 `mte_flen_enc_state_init` (C)

```
mte_status mte_flen_enc_state_init(MTE_HANDLE state, mte_drbgs drbg, uint32_t
tok_bytes, mte_verifiers verifiers, uint32_t fixed_bytes)
```

Initialize the encoder state given the DRBG algorithm, token size in bytes, verifiers algorithm, and fixed length to use. Returns the `status`. Use this function when an MTE-provided DRBG is going to be used.

The state buffer must be of sufficient length to hold the encoder state. See `mte_flen_enc_state_bytes()`.

Buildtime options: the `drbg`, `tok_bytes`, and `verifiers` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_flen_enc_state_init_d()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`fixed_bytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

9.3.1.4 `mte_flen_enc_state_init_d` (C)

```
mte_status mte_flen_enc_state_init_d(MTE_HANDLE state, mte_drbg_instantiate i_cb,
mte_drbg_reseed_counter rc_cb, mte_drbg_state_save_bytes ssb_cb,
mte_drbg_state_save ss_cb, mte_drbg_state_restore sr_cb, mte_drbg_generate g_cb,
mte_drbg_uninstantiate u_cb, void *drbg_state, uint32_t tok_bytes, mte_verifiers
verifiers, uint32_t fixed_bytes)
```

Initialize the encoder state given the DRBG callbacks, DRBG state pointer, token size in bytes, verifiers algorithm, and fixed length to use. Returns the `status`. Use this function when an MTE-provided DRBG is not going to be used.

The state buffer must be of sufficient length to hold the encoder state. See `mte_flen_enc_state_bytes_d()`.

The `drbg_state` must point at the external DRBG state and must remain in scope as long as the state will be used.

Buildtime options: the `tok_bytes`, and `verifiers` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`i_cb`: DRBG instantiate callback.
`rc_cb`: DRBG reseed callback. Can be `NULL` if reseed information will not be requested.
`ssb_cb`: DRBG state save bytes callback. Can be `NULL` if state save/restore will not be used.
`ss_cb`: DRBG state save callback. Can be `NULL` if state save will not be used.
`sr_cb`: DRBG state restore callback. Can be `NULL` if state restore will not be used.
`g_cb`: DRBG generate callback.
`u_cb`: DRBG uninstantiate callback.
`drbg_state`: DRBG state buffer.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`fixed_bytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

9.3.1.5 `mte_flen_enc_instantiate` (C)

```
mte_status mte_flen_enc_instantiate(MTE_HANDLE state, mte_drbg_get_entropy_input
ei_cb, void *ei_cb_context, mte_drbg_get_nonce n_cb, void *n_cb_context, const void
*ps, uint32_t ps_bytes)
```

Instantiate the encoder given the entropy input callback/context, nonce callback/context, personalization string, and length of the personalization string in bytes. Returns the status.

`state`: handle to the encoder state.
`ei_cb`: entropy input callback.
`ei_cb_context`: pointer that will be passed through to the `ei_cb` as the context argument.
`n_cb`: nonce callback.
`n_cb_context`: pointer that will be passed through to the `n_cb` as the context argument.
`ps`: personalization string.
`ps_bytes`: length of the personalization string in bytes.

9.3.1.6 `mte_flen_enc_reseed_counter` (C)

```
uint64_t mte_flen_enc_reseed_counter(MTE_CHANDLE state)
```

Returns the encoder's DRBG reseed counter.

`state`: handle to the encoder state.

9.3.1.7 `mte_flen_enc_save_bytes` (C)

```
uint32_t mte_flen_enc_save_bytes(MTE_CHANDLE state)
```

Returns the state save size for raw state. The associated state save function is `mte_flen_enc_state_save()`.

`state`: handle to the encoder state.

9.3.1.8 `mte_flen_enc_save_bytes_b64` (C)

```
uint32_t mte_flen_enc_save_bytes_b64(MTE_CHANDLE state)
```

Returns the state save size for Base64-encoded state. The associated state save function is `mte_flen_enc_state_save_b64()`.

`state`: handle to the encoder state.

9.3.1.9 `mte_flen_enc_state_save` (C)

```
mte_status mte_flen_enc_state_save(MTE_CHANDLE state, void *saved)
```

Saves the encoder state to the given buffer in raw form. The size of the buffer must be `mte_flen_enc_save_bytes()` and that is the length of the raw saved state. Returns the status.

`state`: handle to the encoder state.
`saved`: the buffer to save the state to.

9.3.1.10 `mte_flen_enc_state_save_b64` (C)

```
mte_status mte_flen_enc_state_save_b64(MTE_CHANDLE state, void *saved)
```

Save the encoder state to the given buffer in Base64-encoded form. The size of the buffer must be `mte_flen_enc_save_bytes_b64()` and the encoded saved state is null-terminated. Returns the status.

`state`: handle to the encoder state.
`saved`: the buffer to save the state to.

9.3.1.11 `mte_flen_enc_state_restore` (C)

```
mte_status mte_flen_enc_state_restore(MTE_HANDLE state, const void *saved)
```

Restores the encoder state from the given buffer in raw form. Returns the status.

`state`: handle to the encoder state.
`saved`: the buffer to restore the state from.

9.3.1.12 `mte_flen_enc_state_restore_b64` (C)

```
mte_status mte_flen_enc_state_restore_b64(MTE_HANDLE state, void *saved)
```

Restores the encoder state from the given buffer in Base64-encoded form. Returns the status.

`state`: handle to the encoder state.
`saved`: the buffer to restore the state from. The Base64 is overwritten with the decoded form.

9.3.1.13 `mte_flen_enc_buff_bytes` (C)

```
uint32_t mte_flen_enc_buff_bytes(MTE_CHANDLE state)
```

Returns the encode buffer size in bytes for raw form encoding. The encode buffer provided to `mte_flen_enc_encode()` must be of at least this length.

`state`: handle to the encoder state.

9.3.1.14 `mte_flen_enc_buff_bytes_b64` (C)

```
uint32_t mte_flen_enc_buff_bytes_b64(MTE_CHANDLE state)
```

Returns the encode buffer size in bytes for Base64-encoded form encoding. The encode buffer provided to `mte_flen_enc_encode_b64()` must be of at least this length.

`state`: handle to the encoder state.

9.3.1.15 `mte_flen_enc_encode` (C)

```
mte_status mte_flen_enc_encode(MTE_HANDLE state, mte_message_get_timestamp64 t_cb,
void *t_cb_context, const void *data, uint32_t data_bytes, void *encoded, uint32_t
*encoded_off, uint32_t *encoded_bytes)
```

Encode the given data of the given byte length to the given encode buffer in raw form. Returns the status. Sets `*encoded_off` to the offset in the `encoded` buffer where the encoded version can be found and sets `*encoded_bytes` to the encoded version length in bytes. The encoded version is valid only if `mte_status_success` is returned.

If `data_bytes` is less than the `fixed_bytes` set during initialization, the data is padded at the end with bytes set to the `byte_val_min` value set during initialization before encoding. If `data_bytes` is greater than the `fixed_bytes` set during initialization, it is truncated to that length before encoding.

The timestamp callback and context are used to obtain the timestamp if the timestamp verifier is enabled.

The `encoded` buffer must be of sufficient length to hold the encoded version. See `mte_flen_enc_buff_bytes()`.

`state`: handle to the encoder state.
`t_cb`: timestamp callback. If the timestamp verifier is not enabled, this can be `NULL`.
`t_cb_context`: pointer that will be passed through to the `t_cb` as the context argument.
`data`: the data to encode. It must contain only byte values allowed by the input range.
`data_bytes`: the length of the data to be encoded (in bytes).
`encoded`: the buffer to write the encoded version of the input to.
`encoded_off`: pointer to the offset of the encoded version.
`encoded_bytes`: pointer to the length of the encoded version.

9.3.1.16 `mte_flen_enc_encode_b64` (C)

```
mte_status mte_flen_enc_encode_b64(MTE_HANDLE state, mte_message_get_timestamp64
t_cb, void *t_cb_context, const void *data, uint32_t data_bytes, void *encoded,
uint32_t *encoded_off, uint32_t *encoded_bytes)
```

Encode the given data of the given byte length to the given encode buffer in Base64-encoded form. Returns the status. Sets `*encoded_off` to the offset in the `encoded` buffer where the Base64 version can be found and sets `*encoded_bytes` to the encoded version length in bytes. The encoded version is null terminated, but `*encoded_bytes` excludes the null terminator. The encoded version is valid only if `mte_status_success` is returned.

If `data_bytes` is less than the `fixed_bytes` set during initialization, the data is padded at the end with bytes set to the `byte_val_min` value set during initialization before encoding. If `data_bytes` is greater than the `fixed_bytes` set during initialization, it is truncated to that length before encoding.

The timestamp callback and context are used to obtain the timestamp if the timestamp verifier is enabled.

The `encoded` buffer must be of sufficient length to hold the encoded version. See `mte_flen_enc_buff_bytes_b64()`.

`state`: handle to the encoder state.
`t_cb`: timestamp callback. If the timestamp verifier is not enabled, this can be `NULL`.
`t_cb_context`: pointer that will be passed through to the `t_cb` as the context argument.
`data`: the data to encode.
`data_bytes`: the length of the data to be encoded (in bytes).
`encoded`: the buffer to write the encoded version of the input to.
`encoded_off`: pointer to the offset of the encoded version.
`encoded_bytes`: pointer to the length of the encoded version.

9.3.1.17 `mte_flen_enc_uninstantiate` (C)

```
mte_status mte_flen_enc_uninstantiate(MTE_HANDLE state)
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The state must not be used until instantiated again.

`state`: handle to the encoder state.

10 C Support API

10.1 Overview

10.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and destroy it.

To create, use `mte_supp_flen_enc_alloc()` to allocate the encoder and `mte_enc_instantiate()` to instantiate it.

To encode, use `mte_supp_flen_enc_encode()` or `mte_supp_flen_enc_encode_b64()`. Use `mte_supp_buff_free()` to free the encoded version after use.

To destroy, use `mte_supp_flen_enc_free()`.

10.2 Files

The MTE Fixed-Length Add-On uses the core header files documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
mte_supp_flen.h	This defines the Fixed-Length Add-On Support API.

10.3 Functions

10.3.1 Encoder Support

Support functions are declared in `mte_supp_flen.h`.

10.3.1.1 `mte_supp_flen_enc_alloc` (C)

```
MTE_HANDLE mte_supp_flen_enc_alloc(mte_drbgs drbg, uint32_t tok_bytes,
mte_verifiers verifiers, uint32_t fixed_bytes)
```

Create an encoder given the DRBG algorithm, token size in bytes, and verifiers algorithm. Returns a handle to the encoder on success or `NULL` on failure. You must call `mte_supp_flen_enc_free()` when you are done with the encoder.

Buildtime options: the `drbg`, `tok_bytes`, and `verifiers` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`fixed_bytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

10.3.1.2 `mte_supp_flen_enc_state_save` (C)

```
void *mte_supp_flen_enc_state_save(MTE_CHANDLE encoder, uint32_t *bytes)
```

Saves the encoder state to the given buffer in raw form. Returns `NULL` on failure; otherwise returns the saved state and sets `*bytes` to the length of the saved state. You must call `mte_supp_buff_free()` when you are done with the saved state.

`encoder`: handle to the encoder state.
`bytes`: pointer to the length of the saved state.

10.3.1.3 `mte_supp_flen_enc_state_save_b64` (C)

```
char *mte_flen_enc_state_save_b64(MTE_CHANDLE encoder, uint32_t *bytes)
```

Saves the encoder state to the given buffer in Base64-encoded form. Returns `NULL` on failure; otherwise returns the saved state and sets `*bytes` to the length of the saved state. You must call `mte_supp_buff_free()` when you are done with the saved state.

`encoder`: handle to the encoder state.
`bytes`: pointer to the length of the saved state.

10.3.1.4 `mte_supp_flen_enc_encode` (C)

```
void *mte_supp_flen_enc_encode(MTE_HANDLE encoder, mte_message_get_timestamp64
t_cb, void *t_cb_context, const void *data, uint32_t data_bytes, uint32_t
*encoded_bytes, mte_status *status)
```

Encode the given data of the given byte length in raw form. Returns the encoded version on success or `NULL` on failure. Sets `*encoded_bytes` to the encoded version length in bytes. Sets `*status` to the status. The encoded version is valid only if `*status == mte_status_success`. You must call `mte_supp_buff_free()` when you are done with the encoded version.

If `data_bytes` is less than the `fixed_bytes` set during initialization, the data is padded at the end with bytes set to the `byte_val_min` value set during initialization before encoding. If `data_bytes` is greater than the `fixed_bytes` set during initialization, it is truncated to that length before encoding.

The timestamp callback and context are used to obtain the timestamp if the timestamp verifier is enabled.

`encoder`: handle to the encoder state.
`t_cb`: timestamp callback. If the timestamp verifier is not enabled, this can be `NULL`.
`t_cb_context`: pointer that will be passed through to the `t_cb` as the context argument.
`data`: the data to encode. It must contain only byte values allowed by the input range.
`data_bytes`: the length of the data to be encoded (in bytes).
`encoded_bytes`: pointer to the length of the encoded version.
`status`: pointer to the status.

10.3.1.5 `mte_supp_flen_enc_encode_b64` (C)

```
char *mte_supp_flen_enc_encode_b64(MTE_HANDLE encoder, mte_message_get_timestamp64
t_cb, void *t_cb_context, const void *data, uint32_t data_bytes, uint32_t
*encoded_bytes, mte_status *status)
```

Encode the given data of the given byte length in Base64-encoded form. Returns the encoded version on success or `NULL` on failure. Sets `*encoded_bytes` to the encoded version length in bytes. Sets `*status` to the status. The encoded version is valid only if `*status == mte_status_success`. You must call `mte_supp_buff_free()` when you are done with the encoded version.

If `data_bytes` is less than the `fixed_bytes` set during initialization, the data is padded at the end with bytes set to the `byte_val_min` value set during initialization before encoding. If `data_bytes` is greater than the `fixed_bytes` set during initialization, it is truncated to that length before encoding.

The timestamp callback and context are used to obtain the timestamp if the timestamp verifier is enabled.

`encoder`: handle to the encoder state.
`t_cb`: timestamp callback. If the timestamp verifier is not enabled, this can be `NULL`.
`t_cb_context`: pointer that will be passed through to the `t_cb` as the context argument.
`data`: the data to encode.
`data_bytes`: the length of the data to be encoded (in bytes).
`encoded_bytes`: pointer to the length of the encoded version.
`status`: pointer to the status.

10.3.1.6 `mte_supp_flen_enc_free` (C)

```
mte_status mte_supp_flen_enc_free(MTE_HANDLE encoder)
```

Free an encoder allocated with `mte_supp_flen_enc_alloc()`. Uninstantiates the encoder before freeing, so the encoder's DRBG state is zeroized. Returns the status.

`encoder`: handle to the encoder state.

11 C++ Language Interface

11.1 Overview

11.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and destroy it.

To create, use an `MteFlenEnc` constructor to create an encoder object and one of the `MteFlenEnc::instantiate()` overloads to instantiate it.

To encode, use any of the `MteFlenEnc::encode()` or `MteFlenEnc::encodeB64()` overloads.

To destroy, cause the `MteFlenEnc` destructor to be invoked (e.g., by going out of scope if on the stack or `delete` if dynamically allocated).

11.2 Files

The MTE Fixed-Length Add-On uses the core header files (other than `MteEnc.h`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteFlenEnc.h	The MteFlenEnc class declaration.

The MTE Fixed-Length Add-On uses the core source files (other than `MteEnc.cpp`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteFlenEnc.cpp	The MteFlenEnc class definition.

11.3 Types

11.3.1 Classes

11.3.1.1 `MteFlenEnc`

Defines encoder-specific methods.

11.4 Functions

11.4.1 Encoder

Encoder-specific functions are declared in `MteFlenEnc.h`.

11.4.1.1 `MteFlenEnc` Constructor (default options) (CPP)

```
MteFlenEnc(size_t fixedBytes)
```

Constructor using default options and the fixed length in bytes. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MteBase::ourDefault`* constants.

`fixedBytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

11.4.1.2 `MteFlenEnc` Constructor (runtime options) (CPP)

```
MteFlenEnc(mte_drbgs drbg, size_t tokBytes, mte_verifiers verifiers, size_t
fixedBytes)
```

Constructor taking the DRBG algorithm, token size in bytes, verifiers algorithm, and fixed length in bytes.

Buildtime options: the `drbg`, `tokBytes`, and `verifiers` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`fixedBytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

11.4.1.3 `MteFlenEnc` Destructor (CPP)

```
virtual ~MteFlenEnc()
```

Destructor. The `uninstantiate()` method is called.

11.4.1.4 `MteFlenEnc::instantiate(void *)` (CPP)

```
mte_status instantiate(const void *ps, size_t psBytes)
```

Instantiate the encoder given the personalization string, and length of the personalization string in bytes. The `entropyCallback()` and `nonceCallback()` virtual functions will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.
`psBytes`: length of the personalization string in bytes.

11.4.1.5 `MteFlenEnc::instantiate(std::string)` (CPP)

```
mte_status instantiate(const std::string& ps)
```

Calls `instantiate(ps.data(), ps.size())`. For interoperability, it is assumed the string is UTF-8 encoded.

`ps`: personalization string.

11.4.1.6 `MteFlenEnc::getReseedCounter` (CPP)

```
uint64_t getReseedCounter() const
```

Returns the encoder's DRBG reseed counter.

11.4.1.7 `MteFlenEnc::saveState` (CPP)

```
const void *saveState(size_t& stateBytes)
```

Returns the saved state in raw form and sets `stateBytes` to the length of the saved state in bytes. On error, `NULL` is returned.

`stateBytes`: the length of the saved state.

11.4.1.8 `MteFlenEnc::saveStateB64` (CPP)

```
char *saveStateB64()
```

Returns the saved state in Base64-encoded form. On error, `NULL` is returned.

11.4.1.9 `MteFlenEnc::restoreState` (CPP)

```
mte_status restoreState(const void *saved)
```

Restores the encoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

11.4.1.10 `MteFlenEnc::restoreStateB64` (CPP)

```
mte_status restoreStateB64(const char *saved)
```

Restore the encoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

11.4.1.11 `MteFlenEnc::getBuffBytes` (CPP)

```
size_t getBuffBytes() const
```

Returns the raw encode buffer size in bytes.

11.4.1.12 `MteFlenEnc::getBuffBytesB64` (CPP)

```
size_t getBuffBytesB64() const
```

Returns the Base64 encode buffer size in bytes.

11.4.1.13 `MteFlenEnc::encode(void *)` (CPP)

```
const void *encode(const void *data, size_t dataBytes, size_t& encodedBytes,
mte_status& status)
```

Encode the given data of the given byte length in raw form. Returns the encoded version and sets `encodedBytes` to the length of the encoded data in bytes and `status` to the status. The encoded version is valid only if `status == mte_status_success`.

If `dataBytes` is less than the `fixedBytes` set during construction, the data is padded at the end with bytes set to the `byteValMin` value set during construction before encoding. If `dataBytes` is greater than the `fixedBytes` set during construction, it is truncated to that length before encoding.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.
`dataBytes`: the length of the data to be encoded (in bytes).
`encodedBytes`: the length of the encoded version.
`status`: the status.

11.4.1.14 `MteFlenEnc::encodeB64(void *)` (CPP)

```
const char *encodeB64(const void *data, size_t dataBytes, mte_status& status)
```

Encode the given data of the given byte length in Base64-encoded form. Returns the encoded version and sets `status` to the status. The encoded version is valid only if `status == mte_status_success`.

If `dataBytes` is less than the `fixedBytes` set during construction, the data is padded at the end with bytes set to the `byteValMin` value set during construction before encoding. If `dataBytes` is greater than the `fixedBytes` set during construction, it is truncated to that length before encoding.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.
`dataBytes`: the length of the data to be encoded (in bytes).
`status`: the status.

11.4.1.15 `MteFlenEnc::encode(std::string)` (CPP)

```
const void *encode(const std::string& str, size_t& encodedBytes, mte_status&
status)
```

Calls `encode(str.data(), str.size(), encodedBytes, status)`. For interoperability, it is assumed the string is UTF-8 encoded.

`str`: the string to encode.
`encodedBytes`: the length of the encoded version.
`status`: the status.

11.4.1.16 `MteFlenEnc::encodeB64(std::string)` (CPP)

```
const char *encodeB64(const std::string& str, mte_status& status)
```

Calls `encodeB64(str.data(), str.size(), status)`. For interoperability, it is assumed the string is UTF-8 encoded.

`str`: the string to encode.
`status`: the status.

11.4.1.17 `MteFlenEnc::encode` (external buffers) (CPP)

```
mte_status encode(const void *data, size_t dataOff, size_t dataBytes, void
*encoded, size_t& encOff, size_t& encBytes)
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in raw form. Returns the status. Adjusts `encOff` to the offset of the encoded version in the `encoded` buffer. Sets `encBytes` to the encoded length in bytes. The `encoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytes()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the encoded data.
`encBytes`: set to the length of the encoded data in bytes.

11.4.1.18 `MteFlenEnc::encodeB64` (external buffers) (CPP)

```
mte_status encodeB64(const void *data, size_t dataOff, size_t dataBytes, void
*encoded, size_t& encOff, size_t& encBytes)
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in Base64-encoded form. Returns the status. Adjusts `encOff` to the offset of the encoded version in the `encoded` buffer. Sets `encBytes` to the encoded length in bytes. The `encoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytesB64()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the encoded data.
`encBytes`: set to the length of the encoded data in bytes.

11.4.1.19 `MteFlenEnc::uninstantiate` (CPP)

```
mte_status uninstantiate()
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

12 C# Language Interface

12.1 Overview

12.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and zeroize it.

To create, use an `MteFlenEnc` constructor to create an encoder object and one of the `MteFlenEnc.Instantiate()` overloads to instantiate it.

To encode, use any of the `MteFlenEnc.Encode()` or `MteFlenEnc.EncodeB64()` overloads.

To zeroize, call `MteFlenEnc.Uninstantiate()`. The destructor will also do this, but C# cannot guarantee when that will happen, if at all, so for security it should be done explicitly.

12.2 Files

The MTE Fixed-Length Add-On uses the core source files (other than `MteEnc.cs`) documented in the MTE Core Developer's Guide, as well as the following:

File	Description
IMteFlenEnc.cs	Interface of the MteFlenEnc class.
MteDelegatesFlenEnc.cs	The delegates for MteFlenEnc.
MteFlenEnc.cs	The MteFlenEnc class.
MteInteropFlenEnc.cs	The MteInterop partial class for MteFlenEnc.

12.3 Types

12.3.1 Classes

12.3.1.1 `MteFlenEnc`

Defines encoder-specific methods.

12.4 Functions

12.4.1 Encoder

Encoder-specific functions are declared in `MteFlenEnc.cs`.

12.4.1.1 `MteFlenEnc` Constructor (default options) (CS)

```
public MteFlenEnc(int fixedBytes)
```

Constructor using default options and the token size in bytes, verifiers algorithm, and fixed length in bytes. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MteBase.ourDefault`* constants. The default interop is used.

`fixedBytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

12.4.1.2 `MteFlenEnc` Constructor (interop, default options) (CS)

```
public MteFlenEnc(MteInterop interop, int fixedBytes)
```

Constructor using default options, a provided interop, and fixed length in bytes. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MteBase.ourDefault`* constants.

`interop`: the interop object to use.
`fixedBytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

12.4.1.3 `MteFlenEnc` Constructor (runtime options) (CS)

```
public MteFlenEnc(MteDrbgs drbg, int tokBytes, MteVerifiers verifiers, int
fixedBytes)
```

Constructor taking the DRBG algorithm, token size in bytes, verifiers algorithm, and fixed length in bytes. The default interop is used.

Buildtime options: the `drbg`, `tokBytes`, and `verifiers` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs.mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`fixedBytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

12.4.1.4 `MteFlenEnc` Constructor (interop, runtime options) (CS)

```
public MteFlenEnc(MteInterop interop, MteDrbgs drbg, int tokBytes, MteVerifiers
verifiers, int fixedBytes)
```

Constructor taking a provided interop, the DRBG algorithm, token size in bytes, verifiers algorithm, and fixed length in bytes.

Buildtime options: the `drbg`, `tokBytes`, and `verifiers` arguments must match the buildtime-chosen options.

`interop`: the interop object to use.
`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs.mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`fixedBytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

12.4.1.5 `MteFlenEnc` Destructor (CS)

```
~MteFlenEnc()
```

Destructor. The `Uninstantiate()` method is called.

12.4.1.6 `MteFlenEnc.Instantiate` (byte[]) (CS)

```
public MteStatus Instantiate(byte[] ps)
```

Instantiate the encoder given the personalization string. The `EntropyCallback()` and `NonceCallback()` virtual functions will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

12.4.1.7 `MteFlenEnc.Instantiate` (`string`) (CS)

```
public MteStatus Instantiate(string ps)
```

Calls `Instantiate()` with the string as an array of UTF-8 encoded bytes.

`ps`: personalization string.

12.4.1.8 `MteFlenEnc.GetReseedCounter` (CS)

```
public UInt64 GetReseedCounter()
```

Returns the encoder's DRBG reseed counter.

12.4.1.9 `MteFlenEnc.SaveState` (CS)

```
public byte[] SaveState()
```

Returns the saved state in raw form and sets `stateBytes` to the length of the saved state in bytes. Returns `null` on error.

12.4.1.10 `MteFlenEnc.SaveStateB64` (CS)

```
public string SaveStateB64()
```

Returns the saved state in Base64-encoded form. Returns `null` on error.

12.4.1.11 `MteFlenEnc.RestoreState` (CS)

```
public MteStatus RestoreState(byte[] saved)
```

Restores the encoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

12.4.1.12 `MteFlenEnc.RestoreStateB64` (CS)

```
public MteStatus RestoreStateB64(string saved)
```

Restores the encoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

12.4.1.13 `MteFlenEnc.GetBuffBytes` (CS)

```
public int GetBuffBytes()
```

Returns the raw encode buffer size in bytes.

12.4.1.14 `MteFlenEnc.GetBuffBytesB64` (CS)

```
public int GetBuffBytesB64()
```

Returns the Base64 encode buffer size in bytes.

12.4.1.15 `MteFlenEnc.Encode` (`byte[]`) (CS)

```
public byte[] Encode(byte[] data, out MteStatus status)
```

Encode the given data in raw form. Returns the encoded version and sets `status` to the status. The encoded version is valid only if `status == MteStatus.mte_status_success`.

If `data.Length` is less than the `fixedBytes` set during construction, the data is padded at the end with bytes set to the `byteValMin` value set during construction before encoding. If `data.Length` is greater than the `fixedBytes` set during construction, it is truncated to that length before encoding.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.
`status`: the status.

12.4.1.16 `MteFlenEnc.EncodeB64` (`byte[]`) (CS)

```
public string EncodeB64(byte[] data, out MteStatus status)
```

Encode the given data in Base64-encoded form. Returns the encoded version and sets `status` to the status. The encoded version is valid only if `status == MteStatus.mte_status_success`.

If `data.Length` is less than the `fixedBytes` set during construction, the data is padded at the end with bytes set to the `byteValMin` value set during construction before encoding. If `data.Length` is greater than the `fixedBytes` set during construction, it is truncated to that length before encoding.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.
`status`: the status.

12.4.1.17 `MteFlenEnc.Encode` (`string`) (CS)

```
public byte[] Encode(string str, out MteStatus status)
```

Calls `Encode()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.
`status`: the status.

12.4.1.18 `MteFlenEnc.EncodeB64` (`string`) (CS)

```
public string EncodeB64(string str, out MteStatus status)
```

Calls `EncodeB64()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.
`status`: the status.

12.4.1.19 `MteFlenEnc.Encode` (external buffers) (CS)

```
public MteStatus Encode(byte[] data, int dataOff, int dataBytes, byte[] encoded,
ref int encOff, out int encBytes)
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in raw form. Returns the status. Adjusts `encOff` to the offset of the encoded version in the `encoded` buffer. Sets `encBytes` to the encoded length in bytes. The `encoded` buffer must have sufficient length remaining after the offset. Use `GetBuffBytes()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the encoded data.
`encBytes`: set to the length of the encoded data in bytes.

12.4.1.20 `MteFlenEnc.EncodeB64` (external buffers) (CS)

```
public MteStatus EncodeB64(byte[] data, int dataOff, int dataBytes, byte[] encoded,
ref int encOff, out int encBytes)
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in Base64-encoded form. Returns the status. Adjusts `encOff` to the offset of the encoded version in the `encoded` buffer. Sets `encBytes` to the encoded length in bytes. The `encoded` buffer must have sufficient length remaining after the offset. Use `GetBuffBytesB64()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the encoded data.
`encBytes`: set to the length of the encoded data in bytes.

12.4.1.21 `MteFlenEnc.Uninstantiate` (CS)

```
public MteStatus Uninstantiate()
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

13 Go Language Interface

13.1 Overview

13.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and destroy it.

To create, use `NewFlenEncDef()` or `NewFlenEncOpt()` to create the encoder object and `(*FlenEnc)Instantiate()` or `(*FlenEnc)InstantiateStr()` to instantiate it.

To encode, use `(*FlenEnc)Encode()`, `(*FlenEnc)EncodeB64()`, `(*FlenEnc)EncodeStr()`, or `(*FlenEnc)EncodeStrB64()`.

To destroy, call `(*FlenEnc)Destroy()`. The Go objects hold on to some global resources, so it is recommended to destroy the object to release those. The destroy method also uninstantiates the encoder for security.

13.2 Files

The MTE Fixed-Length Add-On uses the core source files (other than `MteEnc.go`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteFlenEnc.go	The MteFlenEnc structure and functions.

13.3 Types

13.3.1 Structures

13.3.1.1 ***`FlenEnc`***

Defines encoder-specific data structure.

13.4 Functions

13.4.1 Encoder

Encoder-specific functions are declared in `MteFlenEnc.go`.

13.4.1.1 `NewFlenEncDef` (Go)

```go
func NewFlenEncDef(fixedBytes int) *FlenEnc
```

Creates a new `FlenEnc` structure using default options and the fixed length in bytes. It must be destroyed with `Destroy()` when finished to free resources.

`fixedBytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

13.4.1.2 `NewFlenEncOpt` (Go)

```go
func NewFlenEncOpt(drbg Drbgs, tokBytes int, verifiers Verifiers, fixedBytes int)
*FlenEnc
```

Creates a new `FlenEnc` structure using the DRBG algorithm, token size in bytes, verifiers algorithm, and fixed length in bytes. It must be destroyed with `Destroy()` when finished to free resources.

Buildtime options: the `drbg`, `tokBytes`, and `verifiers` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `Drbgs_mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`fixedBytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

13.4.1.3 `(*FlenEnc)Destroy` (Go)

```go
func (enc *FlenEnc) Destroy()
```

Releases resources. The `Uninstantiate()` method is called.

13.4.1.4 `(*FlenEnc)Instantiate` (Go)

```go
func (enc *FlenEnc) Instantiate(ps []byte) Status
```

Instantiate the encoder given the personalization string. The `EntropyCallback()` and `NonceCallback()` functions defined for the current context set with `ContextStore()` will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

13.4.1.5 `(*FlenEnc)InstantiateStr` (Go)

```go
func (enc *FlenEnc) Instantiate(ps string) Status
```

Calls `Instantiate()` with the string as an array of UTF-8 encoded bytes.

`ps`: personalization string.

13.4.1.6 `(*FlenEnc)GetReseedCounter` (Go)

```go
func (enc *FlenEnc) GetReseedCounter() uint64
```

Returns the encoder's DRBG reseed counter.

13.4.1.7 `(*FlenEnc)SaveState` (Go)

```go
func (enc *FlenEnc) SaveState() []byte
```

Returns the saved state in raw form. Returns `nil` on error.

13.4.1.8 `(*FlenEnc)SaveStateB64` (Go)

```go
func (enc *FlenEnc) SaveStateB64() string
```

Returns the saved state in Base64-encoded form. Returns `nil` on error.

13.4.1.9 `(*FlenEnc)RestoreState` (Go)

```go
func (enc *FlenEnc) RestoreState(saved []byte) Status
```

Restores the decoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

13.4.1.10 `(*FlenEnc)RestoreStateB64` (Go)

```go
func (enc *FlenEnc) RestoreStateB64(saved string) Status
```

Restores the decoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

13.4.1.11 `(*FlenEnc)Encode` (Go)

```go
func (enc *FlenEnc) Encode(data []byte) ([]byte, Status)
```

Encode the given data in raw form. Returns the encoded version and status. The encoded version is valid only if `status == Status_mte_status_success`.

If `len(data)` is less than the `fixedBytes` set during construction, the data is padded at the end with bytes set to the `byteValMin` value set during construction before encoding. If `len(data)` is greater than the `fixedBytes` set during construction, it is truncated to that length before encoding.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.

13.4.1.12 `(*FlenEnc)EncodeB64` (Go)

```go
func (enc *FlenEnc) EncodeB64(data []byte) (string, Status)
```

Encode the given data in Base64-encoded form. Returns the encoded version and status. The encoded version is valid only if `status == Status_mte_status_success`.

If `len(data)` is less than the `fixedBytes` set during construction, the data is padded at the end with bytes set to the `byteValMin` value set during construction before encoding. If `len(data)` is greater than the `fixedBytes` set during construction, it is truncated to that length before encoding.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.

13.4.1.13 `(*FlenEnc)EncodeStr` (Go)

```go
func (enc *FlenEnc) EncodeStr(str string) ([]byte, Status)
```

Calls `Encode()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.

13.4.1.14 `(*FlenEnc)EncodeStrB64` (Go)

```go
func (enc *FlenEnc) EncodeStrB64(str string) (string, Status)
```

Calls `EncodeB64()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.

13.4.1.15 `(*FlenEnc)Uninstantiate` (Go)

```go
func (enc *FlenEnc) Uninstantiate() Status
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

14 Java Language Interface

14.1 Overview

14.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and zeroize it.

To create, use the `MteFlenEnc` constructor to create an encoder object and one of the `MteFlenEnc.instantiate()` overloads to instantiate it.

To encode, use any of the `MteFlenEnc.encode()` or `MteFlenEnc.encodeB64()` overloads.

To zeroize, call `MteFlenEnc.uninstantiate()`.

14.2 Files

The MTE Fixed-Length Add-On uses the core source files (other than `MteEnc.java`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteFlenEnc.java	The MteFlenEnc class.

14.3 Types

14.3.1 Classes

14.3.1.1 `MteFlenEnc`

Defines encoder-specific methods.

14.4 Functions

14.4.1 Encoder

Encoder-specific functions are declared in `MteFlenEnc.java`.

14.4.1.1 `MteFlenEnc` Constructor (default options) (Java)

```
public MteFlenEnc(int fixedBytes)
```

Constructor using default options and the fixed length in bytes. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MteBase.ourDefault`* constants.

`fixedBytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

14.4.1.2 `MteFlenEnc` Constructor (runtime options) (Java)

```
public MteFlenEnc(MteDrbgs drbg, int tokBytes, MteVerifiers verifiers, int
fixedBytes)
```

Constructor taking the DRBG algorithm, token size in bytes, verifiers algorithm, and fixed length in bytes.

Buildtime options: the `drbg`, `tokBytes`, and `verifiers` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs.mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`fixedBytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

14.4.1.3 `MteFlenEnc.instantiate(byte[])` (Java)

```
public MteStatus instantiate(byte[] ps)
```

Instantiate the encoder given the personalization string. The `entropyCallback()` and `nonceCallback()` functions will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

14.4.1.4 `MteFlenEnc.instantiate(String)` (Java)

```
public MteStatus instantiate(String ps)
```

Calls `instantiate()` with the string as an array of UTF-8 encoded bytes.

`ps`: personalization string.

14.4.1.5 `MteFlenEnc.getReseedCounter` (Java)

```
public long getReseedCounter()
```

Returns the encoder's DRBG reseed counter.

14.4.1.6 `MteFlenEnc.saveState` (Java)

```
public byte[] saveState()
```

Returns the saved state in raw form. Returns `null` on error.

14.4.1.7 `MteFlenEnc.saveStateB64` (Java)

```
public String saveStateB64()
```

Returns the saved state in Base64-encoded form. Returns `null` on error.

14.4.1.8 `MteFlenEnc.restoreState` (Java)

```
public MteStatus restoreState(byte[] saved)
```

Restores the encoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

14.4.1.9 `MteFlenEnc.restoreStateB64` (Java)

```
public MteStatus restoreStateB64(String saved)
```

Restores the encoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

14.4.1.10 `MteFlenEnc.getBuffBytes` (Java)

```
public int getBuffBytes()
```

Returns the raw encode buffer size in bytes.

14.4.1.11 `MteFlenEnc.getBuffBytesB64` (Java)

```
public int getBuffBytesB64()
```

Returns the Base64 encode buffer size in bytes.

14.4.1.12 `MteFlenEnc.encode` (`byte[]`) (Java)

```
public ArrStatus encode(byte[] data)
```

Encode the given data in raw form. Returns the encoded version and status. The encoded version is valid only if `status == MteStatus.mte_status_success`.

If `data.length` is less than the `fixedBytes` set during construction, the data is padded at the end with bytes set to the `byteValMin` value set during construction before encoding. If `data.length` is greater than the `fixedBytes` set during construction, it is truncated to that length before encoding.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.

14.4.1.13 `MteFlenEnc.encodeB64` (`byte[]`) (Java)

```
public StrStatus encodeB64(byte[] data)
```

Encode the given data in Base64-encoded form. Returns the encoded version and status. The encoded version is valid only if `status == MteStatus.mte_status_success`.

If `data.length` is less than the `fixedBytes` set during construction, the data is padded at the end with bytes set to the `byteValMin` value set during construction before encoding. If `data.length` is greater than the `fixedBytes` set during construction, it is truncated to that length before encoding.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.

14.4.1.14 `MteFlenEnc.encode` (`String`) (Java)

```
public ArrStatus encode(String str)
```

Calls `encode()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.

14.4.1.15 `MteFlenEnc.encodeB64` (`String`) (Java)

```
public StrStatus encodeB64(String str)
```

Calls `encodeB64()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.

14.4.1.16 `MteFlenEnc.encode` (external buffers) (Java)

```
public OffLenStatus encode(byte[] data, int dataOff, int dataBytes, byte[] encoded,
int encOff)
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in raw form. Returns the offset of the encoded version, length of the encoded version, and status. The `encoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytes()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: the offset to the beginning of the usable buffer area.

14.4.1.17 `MteFlenEnc.encodeB64` (external buffers) (Java)

```
public OffLenStatus encodeB64(byte[] data, int dataOff, int dataBytes, byte[]
encoded, int encOff)
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in Base64-encoded form. Returns the offset of the encoded version, length of the encoded version, and status. The `encoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytesB64()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: the offset to the beginning of the usable buffer area.

14.4.1.18 `MteFlenEnc.uninstantiate` (Java)

```
public MteStatus uninstantiate()
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

15 Objective-C Language Interface

15.1 Overview

15.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and destroy it.

To create, use an `-init`* message initialize an encoder object and an `-instantiate:`* message to instantiate it.

To encode, use an `-encode:`* message.

To destroy, release the object (this may be automatic if using ARC or autorelease).

15.2 Files

The MTE Fixed-Length Add-On uses the core header files (other than `MteEnc.h`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteFlenEnc.h	The MteFlenEnc class declaration.

The MTE Fixed-Length Add-On uses the core source files (other than `MteEnc.m`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteFlenEnc.m	The MteFlenEnc class definition.

15.3 Types

15.3.1 Classes

15.3.1.1 `MteFlenEnc`

Defines encoder-specific methods.

15.4 Functions

15.4.1 Encoder

Encoder-specific functions are declared in `MteFlenEnc.h`.

15.4.1.1 `MteFlenEnc -initWithFixedBytes:` (ObjC)

```
-(id)initWithFixedBytes:(size_t)fixedBytes
```

Initializer using default options and the fixed length in bytes. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MTE_BASE_DEFAULT_*` constants.

`fixedBytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

15.4.1.2 `MteFlenEnc -initWithDrbg:tokBytes:verifiers:fixedBytes:` (ObjC)

```
-(id)initWithDrbg:(mte_drbgs)drbg
         tokBytes:(size_t)tokBytes
        verifiers:(mte_verifiers)verifiers
       fixedBytes:(size_t)fixedBytes
```

Initializer taking the DRBG algorithm, token size in bytes, verifiers algorithm, and fixed length in bytes.

Buildtime options: the `drbg`, `tokBytes`, and `verifiers` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`fixedBytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

15.4.1.3 `MteFlenEnc -dealloc` (ObjC)

```
-(void)dealloc
```

Deallocate. The `-uninstantiate` message is sent.

15.4.1.4 `MteFlenEnc -instantiate:bytes:` (ObjC)

```
-(mte_status)instantiate:(const void *)ps bytes:(size_t)psBytes
```

Instantiate the encoder given the personalization string, and length of the personalization string in bytes. The `-entropyCallbackWithMinEntropy:minLength:maxLength:entropyInput:eiBytes:` and `-nonceCallbackWithMinLength:maxLength:nonce:nBytes:` messages will be sent to get the rest of the seeding material. Returns the status.

`ps`: personalization string.
`psBytes`: length of the personalization string in bytes.

15.4.1.5 `MteFlenEnc -instantiate:` (ObjC)

```
-(mte_status)instantiate:(NSString *)ps
```

Sends `-instantiate:bytes:` with the string as an array of UTF-8 encoded bytes.

`ps`: personalization string.

15.4.1.6 `MteFlenEnc -getReseedCounter` (ObjC)

```
-(uint64_t)getReseedCounter
```

Returns the encoder's DRBG reseed counter.

15.4.1.7 `MteFlenEnc -saveState:` (ObjC)

```
-(const void *)saveState:(size_t *)stateBytes
```

Returns the saved state in raw form and sets `*stateBytes` to the length of the saved state in bytes. Returns `nil` on error.

`stateBytes`: the length of the saved state.

15.4.1.8 `MteFlenEnc -saveStateB64` (ObjC)

```
-(NSString *)saveStateB64
```

Returns the saved state in Base64-encoded form. Returns `nil` on error.

15.4.1.9 `MteFlenEnc -restoreState:` (ObjC)

```
-(mte_status)restoreState:(const void *)saved
```

Restores the encoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

15.4.1.10 `MteFlenEnc -restoreStateB64:` (ObjC)

```
-(mte_status)restoreStateB64:(NSString *)saved
```

Restores the encoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

15.4.1.11 `MteFlenEnc -getBuffBytes` (ObjC)

```
-(size_t)getBuffBytes
```

Returns the raw encode buffer size.

`dataBytes`: the data size in bytes.

15.4.1.12 `MteFlenEnc -getBuffBytesB64` (ObjC)

```
-(size_t)getBuffBytesB64
```

Returns the Base64 encode buffer size.

`dataBytes`: the data size in bytes.

15.4.1.13 `MteFlenEnc -encode:dataBytes:encodedBytes:status:` (ObjC)

```
-(const void *)encode:(const void *)data
            dataBytes:(size_t)dataBytes
         encodedBytes:(size_t *)encodedBytes
               status:(mte_status *)status
```

Encode the given data of the given byte length in raw form. Returns the encoded version and sets `*encodedBytes` to the length of the encoded data in bytes and `*status` to the status. The encoded version is valid only if `*status == mte_status_success`.

If `dataBytes` is less than the `fixedBytes` set during construction, the data is padded at the end with bytes set to the `byte_val_min` value set during construction before encoding. If `dataBytes` is greater than the `fixedBytes` set during construction, it is truncated to that length before encoding.

The timestamp callback message will be sent to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.
`dataBytes`: the length of the data to be encoded (in bytes).
`encodedBytes`: pointer to the length of the encoded version.
`status`: pointer to the status.

15.4.1.14 `MteFlenEnc -encodeB64:dataBytes:status:` (ObjC)

```
-(NSString *)encodeB64:(const void *)data
             dataBytes:(size_t)dataBytes
                status:(mte_status *)status
```

Encode the given data of the given byte length in Base64-encoded form. Returns the encoded version and sets `*status` to the status. The encoded version is valid only if `*status == mte_status_success`.

If `dataBytes` is less than the `fixedBytes` set during construction, the data is padded at the end with bytes set to the `byte_val_min` value set during construction before encoding. If `dataBytes` is greater than the `fixedBytes` set during construction, it is truncated to that length before encoding.

The timestamp callback message will be sent to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.
`dataBytes`: the length of the data to be encoded (in bytes).
`status`: pointer to the status.

15.4.1.15 `MteFlenEnc -encode:encodedBytes:status:` (ObjC)

```
-(const void *)encode:(NSString *)str
         encodedBytes:(size_t *)encodedBytes
               status:(mte_status *)status
```

Sends `-encode:dataBytes:encodedBytes:status:` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.
`encodedBytes`: the length of the encoded version.
`status`: pointer to the status.

15.4.1.16 `MteFlenEnc -encodeB64:status:` (ObjC)

```
-(NSString *)encodeB64:(NSString *)str
                status:(mte_status *)status
```

Sends `-encodeB64:dataBytes:status:` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.
`status`: pointer to the status.

15.4.1.17 `MteFlenEnc -encode:dataOff:dataBytes:encoded:encOff:encBytes` (ObjC)

```
-(mte_status)encode:(const void *)data
            dataOff:(size_t)dataOff
          dataBytes:(size_t)dataBytes
            encoded:(void *)encoded
             encOff:(size_t *)encOff
           encBytes:(size_t *)encBytes
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in raw form. Returns the status. Adjusts `*encOff` to the offset of the encoded version in the `encoded` buffer. Sets `*encBytes` to the encoded length in bytes. The `encoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytes:` to determine the buffer requirement.

The timestamp callback message will be sent to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the encoded data.
`encBytes`: set to the length of the encoded data in bytes.

15.4.1.18 `MteFlenEnc -encodeB64:dataOff:dataBytes:encoded:encOff:encBytes` (ObjC)

```
-(mte_status)encodeB64:(const void *)data
               dataOff:(size_t)dataOff
             dataBytes:(size_t)dataBytes
               encoded:(void *)encoded
                encOff:(size_t *)encOff
              encBytes:(size_t *)encBytes
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in Base64-encoded form. Returns the status. Adjusts `*encOff` to the offset of the encoded version in the `encoded` buffer. Sets `*encBytes` to the encoded length in bytes. The `encoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytesB64:` to determine the buffer requirement.

The timestamp callback message will be sent to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the encoded data.
`encBytes`: set to the length of the encoded data in bytes.

15.4.1.19 `MteFlenEnc -uninstantiate` (ObjC)

```
-(mte_status)uninstantiate
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

16 Python Language Interface

16.1 Overview

16.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and zeroize it.

To create, use the `MteFlenEnc` initializer or `MteFlenEnc.fromdefault()` to create an encoder object and `MteFlenEnc.instantiate()` to instantiate it.

To encode, use `MteFlenEnc.encode()` or `MteFlenEnc.encode_b64()`.

To zeroize, call `MteFlenEnc.uninstantiate()`. The object delete will also do this, but Python cannot guarantee when that will happen, if at all, so for security it should be done explicitly.

16.2 Files

The MTE Fixed-Length Add-On uses the core source files (other than `MteEnc.py`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteFlenEnc.py	The MteFlenEnc class.

16.3 Types

16.3.1 Classes

16.3.1.1 `MteFlenEnc`

Defines encoder-specific methods.

16.4 Functions

16.4.1 Encoder

Encoder-specific functions are declared in `MteFlenEnc.py`.

16.4.1.1 `MteFlenEnc` Initializer (Python)

```
def __init__(self, drbg: MteDrbgs, tok_bytes: int, verifiers: MteVerifiers,
fixed_bytes: int)
```

Constructor taking the DRBG algorithm, token size in bytes, verifiers algorithm, and fixed length in bytes.

Buildtime options: the `drbg`, `tok_bytes`, and `verifiers` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs.mte_drbgs_none`.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`fixed_bytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

16.4.1.2 `MteFlenEnc.fromdefault` (Python)

```
@classmethod
def fromdefault(cls, fixed_bytes: int)
```

Create an encoder using default options and the fixed length in bytes. If the library has buildtime options, they are used; otherwise the options chosen are defined by the `MteBase_default_`* constants.

`fixed_bytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

16.4.1.3 `MteFlenEnc` Delete (Python)

```
def __del__(self)
```

Delete the object. Calls `uninstantiate()`.

16.4.1.4 `MteFlenEnc.instantiate` (Python)

```
def instantiate(self, ps) -> MteStatus
```

Instantiate the decoder given the personalization string. If `instanceof(ps, str)`, it is converted to an array of UTF-8 encoded bytes; otherwise it must be of type `bytes`. The `entropy_callback()` and `nonce_callback()` functions will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

16.4.1.5 `MteFlenEnc.get_reseed_counter` (Python)

```
def get_reseed_counter(self) -> int
```

Returns the encoder's DRBG reseed counter.

16.4.1.6 `MteFlenEnc.save_state` (Python)

```
def save_state(self) -> bytearray
```

Returns the saved state in raw form. Returns an empty array on error.

16.4.1.7 `MteFlenEnc.save_state_b64` (Python)

```
def save_state_b64(self) -> str
```

Returns the saved state in Base64-encoded form. Returns an empty string on error.

16.4.1.8 `MteFlenEnc.restore_state` (Python)

```
def restore_state(self, saved: bytes) -> MteStatus
```

Restores the encoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

16.4.1.9 `MteFlenEnc.restore_state_b64` (Python)

```
def restore_state_b64(self, saved: str) -> MteStatus
```

Restores the encoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

16.4.1.10 `MteFlenEnc.encode` (Python)

```
def encode(self, data) -> (bytearray, MteStatus)
```

Encode the given data in raw form. If `instanceof(data, str)`, it is converted to an array of UTF-8 encoded bytes; otherwise it must be of type `bytes`. Returns the encoded version and status. The encoded version is valid only if `status == MteStatus.mte_status_success`.

If `len(data)` is less than the `fixed_bytes` set during construction, the data is padded at the end with bytes set to the `byte_val_min` value set during construction before encoding. If `len(data)` is greater than the `fixed_bytes` set during construction, it is truncated to that length before encoding.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.

16.4.1.11 `MteFlenEnc.encode_b64` (Python)

```
def encode_b64(self, data) -> (str, MteStatus)
```

Encode the given data in Base64-encoded form. If `instanceof(data, str)`, it is converted to an array of UTF-8 encoded bytes; otherwise it must be of type `bytes`. Returns the encoded version and status. The encoded version is valid only if `status == MteStatus.mte_status_success`.

If `len(data)` is less than the `fixed_bytes` set during construction, the data is padded at the end with bytes set to the `byte_val_min` value set during construction before encoding. If `len(data)` is greater than the `fixed_bytes` set during construction, it is truncated to that length before encoding.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.

16.4.1.12 `MteFlenEnc.uninstantiate` (Python)

```
def uninstantiate(self) -> MteStatus
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

17 Swift Language Interface

17.1 Overview

17.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and destroy it.

To create, use an `MteFlenEnc` initializer to create an encoder object and one of the `MteFlenEnc.instantiate()` overloads to instantiate it.

To encode, use any of the `MteFlenEnc.encode()` or `MteFlenEnc.encodeB64()` overloads.

To destroy, cause the `MteFlenEnc` deinitializer to be invoked (e.g., by removing the last strong reference to the object).

17.2 Files

The MTE Fixed-Length Add-On uses the core source files (other than `MteEnc.swift`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteFlenEnc.swift	The MteFlenEnc class.

17.3 Types

17.3.1 Classes

17.3.1.1 `MteFlenEnc`

Defines encoder-specific methods.

17.4 Functions

17.4.1 Encoder

Encoder-specific functions are declared in `MteFlenEnc.swift`.

17.4.1.1 `MteFlenEnc` Initializer (default options) (Swift)

```
public convenience init(_ fixedBytes: Int) throws
```

Initializer using default options and the fixed length in bytes. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MteBase.ourDefault`* constants.

`fixedBytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

17.4.1.2 `MteFlenEnc` Initializer (runtime options) (Swift)

```
public init(_ drbg: mte_drbgs, _ tokBytes: Int, _ verifiers: mte_verifiers, _
fixedBytes: Int) throws
```

Initializer taking the DRBG algorithm, token size in bytes, verifiers algorithm, and fixed length in bytes.

Buildtime options: the `drbg`, `tokBytes`, and `verifiers` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`fixedBytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

17.4.1.3 `MteFlenEnc` Deinitializer (Swift)

```
deinit
```

Destructor. The `uninstantiate()` method is called.

17.4.1.4 `MteFlenEnc.instantiate(_[UInt8])` (Swift)

```
public func instantiate(_ ps: [UInt8]) -> mte_status
```

Instantiate the encoder given the personalization string. The `entropyCallback()` and `nonceCallback()` functions will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

17.4.1.5 `MteFlenEnc.instantiate(_String)` (Swift)

```
public func instantiate(_ ps: String) -> mte_status
```

Calls `instantiate()` with the string as an array of UTF-8 encoded bytes.

`ps`: personalization string.

17.4.1.6 `MteFlenEnc.getReseedCounter` (Swift)

```
public func getReseedCounter() -> UInt64
```

Returns the encoder's DRBG reseed counter.

17.4.1.7 `MteFlenEnc.saveState` (Swift)

```
public func saveState() -> [UInt8]?
```

Returns the saved state in raw form. Returns `nil` on error.

17.4.1.8 `MteFlenEnc.saveStateB64` (Swift)

```
public func saveStateB64() -> String?
```

Returns the saved state in Base64-encoded form. Returns `nil` on error.

17.4.1.9 `MteFlenEnc.restoreState` (Swift)

```
public func restoreState(_ saved: [UInt8]) -> mte_status
```

Restores the encoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

17.4.1.10 `MteFlenEnc.restoreStateB64` (Swift)

```
public func restoreStateB64(_ saved: String) -> mte_status
```

Restores the encoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

17.4.1.11 `MteFlenEnc.getBuffBytes` (Swift)

```
public func getBuffBytes() -> Int
```

Returns the raw encode buffer size in bytes.

17.4.1.12 `MteFlenEnc.getBuffBytesB64` (Swift)

```
public func getBuffBytesB64() -> Int
```

Returns the Base64 encode buffer size in bytes.

17.4.1.13 `MteFlenEnc.encode` (`[UInt8]`) (Swift)

```
public func encode(_ data: [UInt8]) -> (encoded: ArraySlice<UInt8>, status:
mte_status)
```

Encode the given data in raw form. Returns the encoded version and `status`. The encoded version is valid only if `status == mte_status_success`.

If `data.count` is less than the `fixedBytes` set during construction, the data is padded at the end with bytes set to the `byteValMin` value set during construction before encoding. If `data.count` is greater than the `fixedBytes` set during construction, it is truncated to that length before encoding.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.

17.4.1.14 `MteFlenEnc.encodeB64` (`[UInt8]`) (Swift)

```
public func encodeB64(_ data: [UInt8]) -> (encoded: String, status: mte_status)
```

Encode the given data in Base64-encoded form. Returns the encoded version and `status`. The encoded version is valid only if `status == mte_status_success`.

If `data.count` is less than the `fixedBytes` set during construction, the data is padded at the end with bytes set to the `byteValMin` value set during construction before encoding. If `data.count` is greater than the `fixedBytes` set during construction, it is truncated to that length before encoding.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.

17.4.1.15 `MteFlenEnc.encode` (`String`) (Swift)

```
public func encode(_ str: String) -> (encoded: ArraySlice<UInt8>, status:
mte_status)
```

Calls `encode()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.

17.4.1.16 `MteFlenEnc.encodeB64` (`String`) (Swift)

```
public func encodeB64(_ str: String) -> (encoded: String, status: mte_status)
```

Calls `encodeB64()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.

17.4.1.17 `MteFlenEnc.encode` (external buffers) (Swift)

```
public func encode(_ data: [UInt8], _ dataOff: Int, _ dataBytes: Int, _ encoded:
inout [UInt8], _ encOff: Int) -> (encOff: Int, encBytes: Int, status: mte_status)
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in raw form. Returns the offset of the encoded version, length of the encoded version, and status. The `encoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytes()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: the offset to the beginning of the usable buffer area.

17.4.1.18 `MteFlenEnc.encodeB64` (external buffers) (Swift)

```
public func encodeB64(_ data: [UInt8], _ dataOff: Int, _ dataBytes: Int, _ encoded:
inout [UInt8], _ encOff: Int) -> (encOff: Int, encBytes: Int, status: mte_status)
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in Base64-encoded form. Returns the offset of the encoded version, length of the encoded version, and status. The `encoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytesB64()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: the offset to the beginning of the usable buffer area.

17.4.1.19 `MteFlenEnc.uninstantiate` (Swift)

```
public func uninstantiate() -> mte_status
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

18 WASM Language Interface

18.1 Overview

18.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, zeroize, and destruct it.

To create, use the `MteFlenEnc` constructor or `MteFlenEnc.fromdefault()` to create an encoder object and `MteFlenEnc.instantiate()` to instantiate it.

To encode, use `MteFlenEnc.encode()`, `MteFlenEnc.encodeB64()`, `MteFlenEnc.encodeStr()`, or `MteFlenEnc.encodeStrB64()`.

To zeroize, call `MteFlenEnc.uninstantiate()`.

To destruct, call `MteFlenEnc.destruct()` to free WASM resources.

18.2 Files

The MTE Fixed-Length Add-On uses the core source files documented in the *MTE Core Developer's Guide*.

18.3 Types

18.3.1 Classes

18.3.1.1 `MteFlenEnc`

Defines encoder-specific methods.

18.4 Functions

18.4.1 Encoder

18.4.1.1 `MteFlenEnc` Constructor (WASM)

```
constructor(wasm: MteWasm, drbg: mte_drbgs | string, tokBytes: number, verifiers:
MteVerifiers | string, fixedBytes: number)
```

Constructor taking the DRBG algorithm, token size in bytes, verifiers algorithm, and fixed length in bytes.

Buildtime options: the `drbg`, `tokBytes`, and `verifiers` arguments must match the buildtime-chosen options.

`wasm`: the instantiated WASM.
`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs.mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`fixedBytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

18.4.1.2 `MteFlenEnc.fromdefault` (WASM)

```
public static fromdefault(wasm: MteWasm, fixedBytes: number): MteFlenEnc
```

Constructor using default options and the fixed length in bytes. If the library has buildtime options, they are used; otherwise the options chosen are defined by the `MteBase.default`* constants.

`wasm`: the instantiated WASM.
`fixedBytes`: the fixed length of the *input* to use. This is not the fixed output size. The input to encode is truncated or padded to this length.

18.4.1.3 `MteFlenEnc.destruct` (WASM)

```
destruct = (): void
```

Destruct. The WASM memory will be freed and the object can no longer be used.

18.4.1.4 `MteFlenEnc.instantiate` (WASM)

```
instantiate = (ps: string): MteStatus
```

Instantiate the decoder given the personalization string. The personalization string is converted to an array of UTF-8 encoded bytes for use. The entropy and nonce will be used, depending how they were set, to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

18.4.1.5 `MteFlenEnc.getReseedCounter` (WASM)

```
getReseedCounter = (): string
```

Returns the encoder's DRBG reseed counter. The string will contain a decimal number which could exceed the range of Javascript's `Number` type.

18.4.1.6 `MteFlenEnc.saveState` (WASM)

```
saveState = (): Uint8Array | null
```

Returns the saved state in raw form and sets `stateBytes` to the length of the saved state in bytes. Returns `null` on error.

18.4.1.7 `MteFlenEnc.saveStateB64` (WASM)

```
saveStateB64 = (): string | null
```

Returns the saved state in Base64-encoded form. Returns `null` on error.

18.4.1.8 `MteFlenEnc.restoreState` (WASM)

```
restoreState = (saved: Uint8Array): MteStatus
```

Restores the encoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

18.4.1.9 `MteFlenEnc.restoreStateB64` (WASM)

```
restoreStateB64 = (saved: string): MteStatus
```

Restores the encoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

18.4.1.10 `MteFlenEnc.encode` (WASM)

```
encode = (data: Uint8Array): MteArrStatus
```

Encode the given data in raw form. Returns the encoded version and status. The encoded version is valid only if `status == MteStatus.mte_status_success`.

If `data.length` is less than the `fixedBytes` set during construction, the data is padded at the end with bytes set to the `byteValMin` value set during construction before encoding. If `data.length` is greater than the `fixedBytes` set during construction, it is truncated to that length before encoding.

The timestamp callback will be called if the timestamp verifier is enabled.

`data`: the data to encode.

18.4.1.11 `MteFlenEnc.encodeB64` (WASM)

```
encodeB64 = (data: Uint8Array): MteStrStatus
```

Encode the given data in Base64-encoded form. Returns the encoded version and status. The encoded version is valid only if `status == MteStatus.mte_status_success`.

If `data.length` is less than the `fixedBytes` set during construction, the data is padded at the end with bytes set to the `byteValMin` value set during construction before encoding. If `data.length` is greater than the `fixedBytes` set during construction, it is truncated to that length before encoding.

The timestamp callback will be called if the timestamp verifier is enabled.

`data`: the data to encode.

18.4.1.12 `MteFlenEnc.encodeStr` (WASM)

```
encodeStr = (str: string): MteArrStatus
```

Calls `encode()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.

18.4.1.13 `MteFlenEnc.encodeStrB64` (WASM)

```
encodeStrB64 = (str: string): MteStrStatus
```

Calls `encodeB64()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.

18.4.1.14 `MteFlenEnc.uninstantiate` (WASM)

```
uninstantiate = (): MteStatus
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

19 Contact Eclypses

For more information, please contact:

info@eclypses.com

www.eclypses.com

+1.719.323.6680

All trademarks of Eclypses Inc. may not be used without Eclypses Inc.'s prior written consent. No license for any use thereof has been granted without express written consent. Any unauthorized use thereof may violate copyright laws, trademark laws, privacy and publicity laws and communications regulations and statutes. The names, images and likeness of the Eclypses logo, along with all representations thereof, are valuable intellectual property assets of Eclypses, Inc. Accordingly, no party or parties, without the prior written consent of Eclypses, Inc., (which may be withheld in Eclypses' sole discretion), use or permit the use of any of the Eclypses trademarked names or logos of Eclypses, Inc. for any purpose other than as part of the address for the Premises, or use or permit the use of, for any purpose whatsoever, any image or rendering of, or any design based on, the exterior appearance or profile of the Eclypses trademarks and or logo(s).